# PREDICTION OF GENETIC DISORDERS USING CHROMOSOME IMAGES WITH DEEP LEARNING AND CONVOLUTIONAL NEURAL NETWORK (CNN)

**THIRD YEAR**

**MINI PROJECT REPORT**

Submitted by


**ELAMATHI C – 24623109**

**JOSLYN J R – 24623115**

**JASMINE EPSIBA A – 24623103**


In partial fulfilment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY IN CSE-**

**ARTIFICIAL INTELLIGENCE**


**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

**SCHOOL OF ENGINEERING**

**VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)**

**CHENNAI – 600 117**


**NOVEMBER 2026**

## BONAFIDE CERTIFICATE

Certified that this project report "**PREDICTION OF GENETIC DISORDERS USING CHROMOSOME IMAGES WITH DEEP LEARNING AND CONVOLUTIONAL NEURAL NETWORK (CNN)**" is the Bonafide work of **ELAMATHI C – 24623109 ,JOSLYN J R – 24623115** and **JASMINE EPSIBA A – 24623103**, who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.


**SIGNATURE OF THE HOD**
**Dr. K. Ulaga Priya M.E., Ph.D**
Associate Professor & Head
Department of AI&ML
School of Engineering VISTAS
Chennai

**SIGNATURE OF THE SUPERVISOR**
**Ms. F. Benasir Begam M.E., (Ph.D)**
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS
Chennai


Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

**INTERNAL EXAMINER**

**EXTERNAL EXAMINER**

# DECLARATION

We hereby declare that the mini project report entitled "**KARYOPREDICT: PREDICTION OF GENETIC DISORDERS USING CHROMOSOME IMAGES WITH DEEP LEARNING AND CONVOLUTIONAL NEURAL NETWORK (CNN )**", submitted to Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence), is a record of original work carried out by us under the guidance of **Ms. Benasir Begam F, M.E, (Ph.D.),** Assistant Professor, Department of Artificial Intelligence and Machine Learning. The work has not been submitted, in part or in full, for the award of any other degree or diploma.

Place: Chennai

Date: 30.10.2026

**ELAMATHI C – 24623109**

**JOSLYN J R – 24623115**

**JASMINE EPSIBA A – 24623103**

## ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our **FOUNDER & CHAIRMAN, Dr. ISHARI K. GANESH** , and it is with great pleasure that we take this opportunity to thank him.We are grateful to our **HEAD OF THE DEPARTMENT, Dr K. Ulaga Priya**, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE, Ms F. Benasir Begam, Assistant Professor, Department of AI&ML**, who has patiently guided us and provided invaluable advice and help.We would like to express our greatest gratitude to our staff members, Librarian, and nonteaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not the least, we wish to thank our parents and friends for their individual support and interest, who have inspired and encouraged us to go our own way; without them we would be unable to complete this project.

**ELAMATHI C – 24623109**
**JOSLYN J R – 24623115**
**JASMINE EPSIBA A – 24623103**

| CHAPTER NO. | TABLE OF CONTENT | PAGE NO. |
|---|---|---|
| | **ABSTRACT** | i |
| | **LIST OF FIGURES** | ii |
| | **ABBREVIATIONS** | iii |
| **1** | **INTRODUCTION** | 1 |
| | 1.1 Overview of the Project | 1 |
| | 1.2 Scope and Objective | 3 |
| **2** | **LITERATURE REVIEW** | 5 |
| | 2.1 Literature Overview | 5 |
| | 2.2 Literature Survey | 5 |
| | 2.3 Research Gap and Project Positioning | 6 |
| **3** | **SYSTEM DESIGN** | 7 |
| | 3.1 Existing System | 7 |
| | 3.2 Proposed System | 7 |
| | 3.3 System Architecture | 7 |
| | 3.4 Hardware Requirements | 8 |
| | 3.5 Software Requirements | 9 |
| **4** | **IMPLEMENTATION** | 10 |
| | 4.1 Methodology | 10 |
| | 4.2 Implementation | 13 |
| | 4.2.1 Dataset Generation and Verification Module | 13 |
| | 4.2.2 Data Loading and Augmentation Module | 14 |
| | 4.2.3 CNN Model Module | 14 |
| | 4.2.4 Training Module | 14 |
| | 4.2.5 Evaluation and Prediction Module | 14 |
| | 4.2.6 Web Application Module | 14 |
| **5** | **CONCLUSION AND FUTURE WORK** | 16 |
| | 5.1 Conclusion | 16 |
| | 5.2 Future Work | 16 |
| | 5.3 Source Code | 17 |
| | 5.4 Results | 30 |
| | **APPENDIX A-CONFERENCE PAPER CERTIFICATES** | 33 |
| | **REFERENCES** | 34 |

## ABSTRACT

Chromosomal abnormalities are a major cause of genetic disorders, and their diagnosis depends on karyotyping, a process in which chromosome images are examined manually by trained cytogeneticists. This manual analysis is time-consuming and depends heavily on individual expertise. This project proposes a deep learning system that analyses chromosome karyotype images with a Convolutional Neural Network (CNN) and predicts one of six classes: normal, Down syndrome (trisomy 21), Edwards syndrome (trisomy 18), Patau syndrome (trisomy 13), Turner syndrome (monosomy X) and Klinefelter syndrome (47,XXY). Because labelled public image sets of abnormal karyotypes are scarce, a six-class dataset of 6,000 karyotypestyle images (1,000 per class) is generated programmatically by a Python script. The script draws the 22 autosome pairs and the sex chromosomes and sets the chromosome count according to the class, for example three copies of chromosome 21 for Down syndrome and a single X for Turner syndrome. The images are split 70/15/15 into training, validation and test sets by template, so that no image leaks across splits. A transfer-learning model based on MobileNetV2, pre-trained on ImageNet, is trained in two phases (the classification head first, then fine-tuning of the last layers) with mild data augmentation, early stopping, learning-rate reduction and checkpointing. The model is evaluated on the held-out test set with accuracy, precision, recall, macro and weighted F1-score, a confusion matrix and training curves. It is integrated into a Flask web application with user registration and login, image upload, prediction with a confidence score, a screening history stored in SQLite and a modelperformance dashboard. The system is an academic decision-support prototype; validation on real patient karyotypes remains future work.



**Keywords:** chromosome images, karyotype, genetic disorder prediction, Convolutional Neural Network, transfer learning, MobileNetV2, Flask, Down syndrome, deep learning.

# LIST OF FIGURES


| FIGURE NO | TITLE | PAGE NO. |
|---|---|---|
| Figure 1 | Proposed system architecture for genetic disorder prediction | 7 |
| Figure 2 | Workflow for generating the dataset | 10 |
| Figure 3 | Sample image of each class in the dataset | 10 |
| Figure 4 | CNN model and two-phase training pipeline | 11 |
| Figure 5 | Login and registration pages | 14 |
| Figure 6 | Karyotype screening dashboard | 15 |
| Figure 7 | Confusion matrix on the test set | 33 |
| Figure 8 | Training vs validation accuracy | 33 |
| Figure 9 | Training vs validation loss | 33 |
| Figure 10 | Per-class F1-score | 34 |


# ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| **AI** | Artificial Intelligence |
| **ML** | Machine Learning |
| **CNN** | Convolutional Neural Network |
| **ReLU** | Rectified Linear Unit |
| **CSRF** | Cross-Site Request Forgery |
| **SQL** | Structured Query Language |
| **TP / TN** | True Positive / True Negative |
| **FP / FN** | False Positive / False Negative |
| **GPU** | Graphics Processing Unit |
| **RAM** | Random Access Memory |
| **SSD** | Solid State Drive |
| **HTML** | HyperText Markup Language |
| **API** | Application Programming Interface |
| **GUI** | Graphical User Interface |
| **CSS** | Cascading Style Sheets |
| **G-banding** | Giemsa banding (chromosome staining) |

# CHAPTER 1. INTRODUCTION

## 1.1 Overview of the Project

### Introduction

Human cells normally contain 46 chromosomes arranged in 23 pairs, which carry the genetic information essential for growth and development. Abnormalities in chromosome number or structure can lead to various genetic disorders. Karyotyping is a standard cytogenetic technique used to identify these abnormalities by staining, photographing, and arranging chromosomes based on their size, centromere position, and banding patterns. The resulting karyogram is examined by a cytogeneticist to detect chromosomal variations. However, manual karyogram preparation and analysis are time-consuming and require considerable expertise. Recent advancements in Artificial Intelligence (AI) and Deep Learning offer opportunities to automate this process. Convolutional Neural Networks (CNNs) are effective image classification techniques that automatically extract relevant features and recognize patterns from image data. This project aims to develop a CNN-based chromosome image classification system that predicts whether an input image belongs to a normal category or one of five selected chromosomal disorder classes. The model is trained using labelled chromosome images and evaluated using performance metrics such as accuracy, precision, recall, and F1-score. The proposed system aims to improve classification efficiency, reduce manual effort, and support preliminary genetic screening. However, it is intended as a decision-support tool and does not replace professional cytogenetic evaluation. Overall, this project demonstrates the potential of deep learning in automating chromosome image analysis and supporting medical diagnostic workflows.

**Table 1.1: Chromosomal disorders considered in this project**

| Class | Typical karyotype | Chromosomal change |
|---|---|---|
| Normal | 46,XX or 46,XY | None (23 pairs) |
| Down syndrome | 47,XX,+21 or 47,XY,+21 | Extra copy of chromosome 21 (trisomy 21) |
| Edwards syndrome | 47,XX,+18 or 47,XY,+18 | Extra copy of chromosome 18 (trisomy 18) |
| Patau syndrome | 47,XX,+13 or 47,XY,+13 | Extra copy of chromosome 13 (trisomy 13) |
| Turner syndrome | 45,X | Only one X chromosome (monosomy X) |
| Klinefelter syndrome | 47,XXY | Two X chromosomes plus one Y in a male |

### Problem Statement

Conventional chromosome analysis has the following limitations:

- **Time-consuming:** Sorting, pairing and reading chromosomes manually takes considerable laboratory time for each case.
- **Dependence on expertise:** Results depend on the experience of the examiner, which can introduce variability between observers.
- **Scarce labelled data:** Openly available chromosome image sets mostly contain normal karyotypes; labelled images of each abnormal syndrome are rarely shared because they are patient data.
- **Narrow focus of existing deep learning work:** Many studies address segmentation, detection or single-chromosome classification rather than predicting the disorder from the image.

The core problem is the lack of an affordable, easily usable pipeline that takes a chromosome image and directly predicts the likely disorder together with a confidence score.

### System Overview

The system consists of three major components:

1. **Dataset and Preprocessing:** A six-class dataset of karyotype images is generated programmatically, split into training, validation and test sets and resized to 224 × 224 pixels.
2. **CNN Classification:** A MobileNetV2 based network, pre-trained on ImageNet and fine-tuned on the karyotype images, classifies each image into one of six classes.

3. **Prediction Output and Web Interface:** The system reports the predicted class with a softmax confidence score and produces evaluation metrics and graphs. A Flask web application provides registration, login, image upload, prediction history and a modelperformance page.

**Advantages of the System**

- **Faster screening support:** An image is classified within a fraction of a second after the model is trained.
- **Consistent output:** The same image always receives the same prediction, which reduces observer-to-observer variability.
- **Transparent evaluation:** Accuracy, precision, recall, F1-score, confusion matrix and training curves are computed from the actual test-set predictions.
- **Low cost:** The pipeline uses open-source tools and a lightweight model that can be trained on a laptop CPU or on a free Google Colab GPU. **Applications**
- **Cytogenetics laboratories:** As a pre-screening aid that highlights images needing closer expert review.
- **Education and training:** To demonstrate how numerical chromosome abnormalities appear in karyotypes.
- **Research:** As a baseline for further work on automated karyotype analysis.

The system is designed as an assistive decision-support tool. It does not replace the judgement of a qualified cytogeneticist or a laboratory diagnosis.

## 1.2 Scope and Objective

**Objectives of the Project**

- To build a CNN-based pipeline for direct classification of chromosome images into six classes: normal, Down, Edwards, Patau, Turner and Klinefelter.
- To overcome the shortage of labelled abnormal images by generating a six-class dataset in which the chromosome counts follow each disorder.
- To train and evaluate the model on separate training, validation and test sets with no overlap between templates.
- To measure performance with accuracy, precision, recall, macro and weighted F1score, confusion matrix and prediction confidence.
- To provide a web interface with registration, login, image upload, prediction history and a model-performance dashboard. **Scope of the Project**

- **Numerical abnormalities only:** The six classes cover trisomy 21, 18 and 13, monosomy X and 47,XXY. Structural abnormalities are outside the scope.
- **Image-level classification:** Each input image is assigned one class.
- **Prototype status:** The system is a research prototype. It has not been clinically validated.

**Expected Outcomes**

- A trained CNN model that separates the six classes on a held-out test set.
- Reproducible scripts for dataset generation, verification, training, evaluation and prediction.
- Evaluation graphs and a classification report generated from real test-set predictions.
- A working web application that connects registration, login, prediction, history and performance metrics.

# CHAPTER 2. LITERATURE REVIEW

## 2.1 Literature Overview

Early automated karyotyping methods relied on hand-crafted geometric features such as chromosome length, centromere position and banding profiles, followed by classical classifiers [3]. With the arrival of deep learning, CNNs replaced hand-crafted features. Zhang et al. [5] classified chromosomes into 23 types with a CNN trained on 10,304 images and reached 91.3% accuracy. Jindal et al. [6] used Siamese networks and straightening techniques to speed up training and prediction. Sharma et al. [7] combined crowdsourced annotation, straightening and a hierarchical deep network for classification of chromosomes from healthy patients.

Later work improved accuracy and robustness. Varifocal-Net [4] uses a global-scale and a local-scale network to classify both the type and the polarity of chromosomes and reported 99.2% accuracy per patient case on 1,909 cases. Wang et al. [3] proposed an extended ResNet with a label feature vector and label redistribution and reported 94.72% accuracy on microscopic images. For detection in whole metaphase cells, Kuo et al. [1] proposed ChromosomeNet and evaluated it on 5,000 metaphase images containing 229,852 chromosomes.

Public data have also become available. Tseng et al. [2] released an open dataset of annotated metaphase cell images, and AutoKary2022 [8] provides more than 27,000 densely annotated chromosome instances from 612 microscopic images. These datasets contain mostly normal karyotypes with chromosome-level labels, which motivates the dataset strategy used in this project.

## 2.2 Literature Survey

**Table 2.1: Summary of related work**

| No. | Author(s) & Year | Title | Methodology | Limitation |
|---|---|---|---|---|
| 1 | Kuo et al., 2024 | ChromosomeNet: Deep Learning-Based Automated Chromosome Detection in Metaphase Cell Images | CNN-based detector evaluated on 5,000 metaphase images | Detects and recognises chromosomes only; no disorder prediction |
| 2 | Tseng et al., 2023 | An Open Dataset of Annotated Metaphase Cell Images for Chromosome Identification | Open dataset with chromosome boxes and 24 classes | Chromosome-level labels only; no syndrome-level labels |
| 3 | Wang et al., 2020 | Extended ResNet and Label Feature Vector Based Chromosome Classification | Extended ResNet, label feature vector and label redistribution; 94.72% accuracy | Single-chromosome classification only |
| **No.** | **Author(s) & Year** | **Title** | **Methodology** | **Limitation** |

| | | | | |
|---|---|---|---|---|
| 4 | Qin et al., 2019 | Varifocal-Net: A Chromosome Classification Approach Using Deep Convolutional Networks | Global and local CNNs with varifocal mechanism; 99.2% per case | Complex two-network design; type and polarity only |
| 5 | Zhang et al., 2018 | Chromosome Classification with Convolutional Neural Network Based Deep Learning | CNN for 23 chromosome types; 91.3% accuracy | Lower accuracy; chromosome type only |
| 6 | Jindal et al., 2017 | Siamese Networks for Chromosome Classification | Siamese networks with straightening | Needs paired inputs; healthy-patient data only |
| 7 | Sharma et al., 2017 | Crowdsourcing for Chromosome Segmentation and Deep Classification | Crowdsourced annotation, straightening and hierarchical deep network | Healthy-patient data only; no disorder prediction |

## 2.3 Research Gap and Project Positioning What

**prior work often assumes**

- The focus is on segmentation, detection or single-chromosome classification rather than disorder-level prediction.
- Datasets contain mostly normal karyotypes, and labelled abnormal syndromes are limited.
- Complex or costly systems that are not easy for small laboratories to adopt. **What this project addresses**
- A CNN-based pipeline that classifies a chromosome image directly into normal or one of five disorders.
- A dataset strategy that generates the disorder classes with the correct chromosome counts and a leak-free split, with the limitations stated openly.
- A transparent evaluation that reports metrics computed from real test-set predictions, with clear outputs (predicted class and confidence).
- An integrated web application in which the prediction, history and performance pages all use the actual model outputs.

# CHAPTER 3. SYSTEM DESIGN

## 3.1 Existing System

In current practice, karyotyping is performed by cytogeneticists who prepare slides, photograph metaphase cells and manually cut, pair and order chromosomes, often with the help of commercial karyotyping software. Published deep learning systems [1]–[7] automate individual steps such as chromosome detection, segmentation or classification of single chromosomes.

The limitations of these systems are:

- Manual steps remain slow and expertise-dependent.
- Most deep learning systems stop at chromosome-level output and do not predict the disorder.
- Training data are mostly normal karyotypes.
- Commercial systems are costly for small laboratories.

## 3.2 Proposed System

The proposed system treats the problem as image classification. An input chromosome image is passed to a CNN that outputs a score for each of the six classes. The class with the highest score is the prediction and that score is reported as the confidence.

- **Six-class output:** normal, Down, Edwards, Patau, Turner and Klinefelter.
- **Transfer learning:** MobileNetV2 pre-trained on ImageNet [9] is fine-tuned on the karyotype images; it is lightweight and works well with limited data.
- **Leak-free evaluation:** Images are split by template into training, validation and test sets, so no template appears in more than one split.
- **Complete reporting:** Accuracy, precision, recall, macro and weighted F1-score, confusion matrix and training curves are generated automatically by evaluate.py.
- **Web application:** A Flask and SQLite application provides authentication, image upload, prediction, history and a model-performance dashboard.

## 3.3 System Architecture

Figure 1 shows the architecture of the proposed system. It has five parts: the image dataset, preprocessing, the CNN model, the Flask backend with its database, and the web interface.

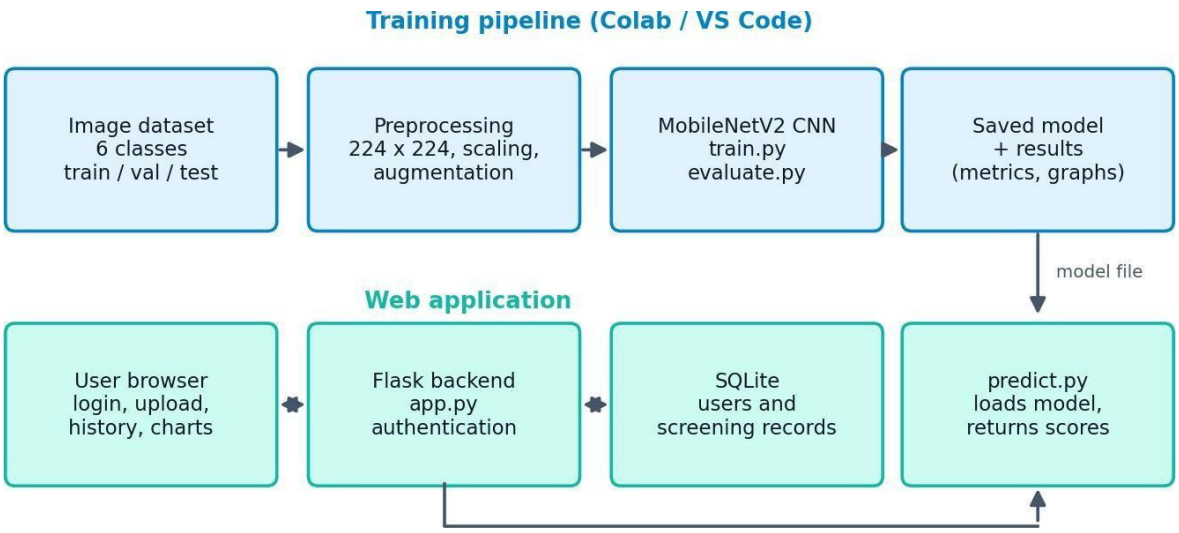



**Figure 1: Proposed system architecture for genetic disorder prediction**

- **Image dataset:** 6,000 images (1,000 per class) are split 70/15/15 into 4,200 training, 900 validation and 900 test images.
- **Preprocessing:** Images are resized to 224 × 224 pixels and scaled inside the model; mild augmentation is applied to the training set only.
- **CNN model:** MobileNetV2 extracts features, followed by global average pooling, dropout, a dense layer and a six-way softmax layer.
- **Flask backend and web interface:** app.py handles registration, login, uploads and prediction through predict.py, and stores users and screening records in SQLite. The interface shows results, history and performance charts.

## 3.4 Hardware Requirements

**Table 3.1: Hardware requirements**

| Component | Specification | Purpose |
|---|---|---|
| **Processor (CPU)** | Intel Core i5 / AMD Ryzen 5 or better | Data loading, augmentation and the web server |
| **GPU** | Optional: NVIDIA GPU or the free Google Colab T4 GPU | Speeds up CNN training |
| **Memory (RAM)** | 8 GB (16 GB recommended) | Holds image batches during training |
| **Storage** | 10 GB free disk space (the dataset needs about 70 MB) | Dataset, model, results and database |

## 3.5 Software Requirements

**Table 3.2: Software requirements**

| Category | Specification |
|---|---|
| **Operating system** | Windows 11 / Ubuntu 22.04 LTS |
| **Programming language** | Python 3.10 - 3.12 (64-bit) |
| **Development environment** | VS Code and Google Colab |
| **Deep learning framework** | TensorFlow 2.x with Keras [10] |
| **Image processing** | Pillow, NumPy |
| **Evaluation and plots** | Scikit-learn [12], Matplotlib |
| **Web application** | Flask [11], SQLite, HTML, CSS, JavaScript, Chart.js |

# CHAPTER 4. IMPLEMENTATION

## 4.1 Methodology

### Step 1: Dataset generation

A Python script, generate_dataset.py, generates the six-class dataset. Each image is a karyotype-style layout of 24 slots (chromosomes 1 to 22, X and Y). Every chromosome is drawn as a banded body with a centromere, with random length, angle, position, brightness, background and noise, and the number of copies placed in each slot follows the class (Table 4.1). Images are saved to disk one at a time, so memory use stays low, and the number of images per class can be set with the --per_class option (1,000 by default). The images are illustrative renderings produced by the script and are not images of real patients; this is a limitation of the work (Section 5.1).

### Step 2: Class definitions

Table 4.1 shows the chromosome pattern drawn for each class. Figure 2 shows the generation workflow and Figure 3 shows one sample image of each class.

**Table 4.1: Chromosome pattern drawn for each class**

| Class | Chromosome pattern | Copies drawn | Resulting karyotype |
|---|---|---|---|
| **Normal** | Two of every chromosome; XX or XY chosen at random | 2 × (1-22) + XX or XY | 46,XX / 46,XY |
| **Down syndrome** | Three copies of chromosome 21 | 3 × chr 21 | 47,+21 |
| **Edwards syndrome** | Three copies of chromosome 18 | 3 × chr 18 | 47,+18 |
| **Patau syndrome** | Three copies of chromosome 13 | 3 × chr 13 | 47,+13 |
| **Turner syndrome** | One X chromosome and no Y | 1 × X | 45,X |
| **Klinefelter syndrome** | Two X chromosomes and one Y | 2 × X + 1 × Y | 47,XXY |

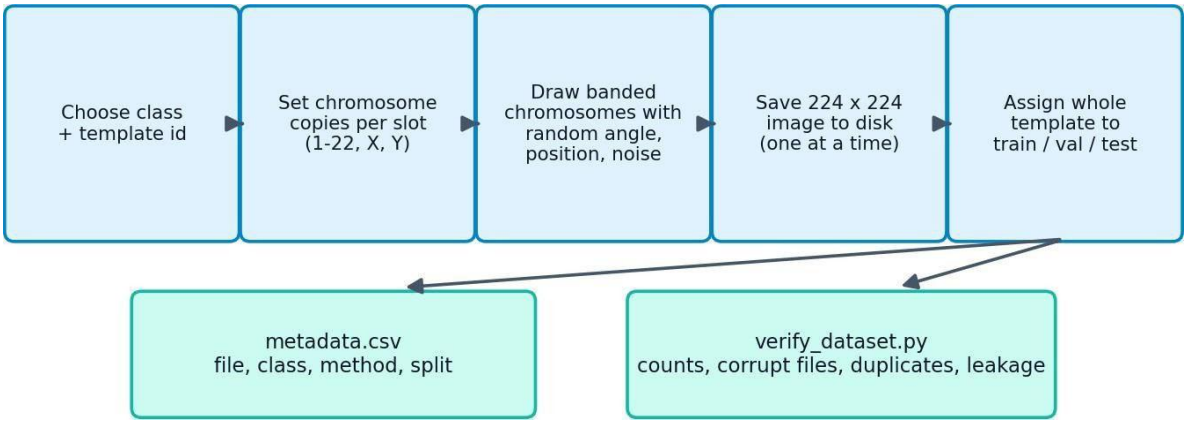



**Figure 2: Workflow for generating the dataset**

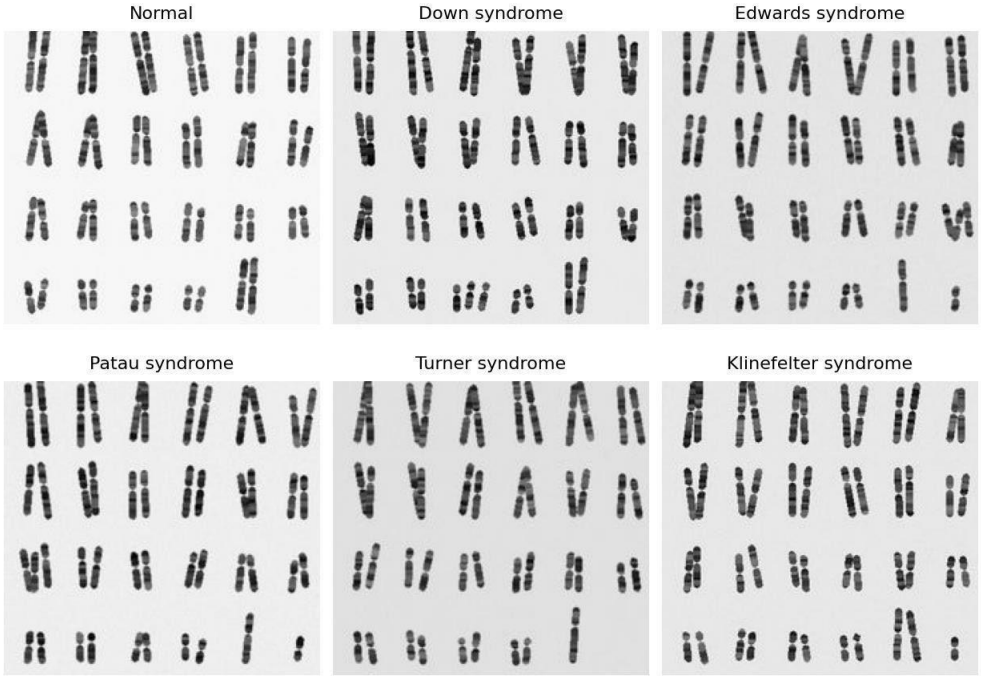



**Figure 3: Sample image of each class in the dataset**

### Step 3: Leak-free split

Every image is generated from its own template, and each template is assigned as a whole to the training (70%), validation (15%) or test (15%) split. This gives 700, 150 and 150 images per class (4,200, 900 and 900 images in total). Because a template can never appear in two splits, no image or near copy leaks across splits. The script verify_dataset.py reports the number of images in every class and split, finds corrupt files, exact duplicates (MD5 hash) and nearduplicates, and confirms that no template is shared between splits. The file metadata.csv records the file name, class, generation method and split of every image.

### Step 4: Preprocessing and augmentation

Images are 224 × 224 pixels. Pixel values are scaled to the range -1 to 1 (x / 127.5 - 1), as MobileNetV2 expects; this scaling is a layer inside the model, so training and prediction use

identical preprocessing. During training only, small random rotation (±3%), translation (±3%), brightness and contrast changes (±10%) are applied. Flips, zoom and cropping are not used, so the number of chromosomes in an image is never changed.

### Step 5: Model and training

A MobileNetV2 [9] base network with ImageNet weights, without its original classifier, is followed by global average pooling, dropout (0.3), a dense layer of 64 units and a six-unit softmax layer (Figure 4). Training has two phases. First the base is frozen and only the new layers are trained for up to 15 epochs at a learning rate of 1e-3. Then the last 30 layers of the base are unfrozen and fine-tuned at 1e-5 for up to 8 epochs. Early stopping ends training when the validation loss stops improving, the learning rate is halved when the validation loss stalls, and the best model by validation accuracy is saved to models/karyopredict_model.keras.

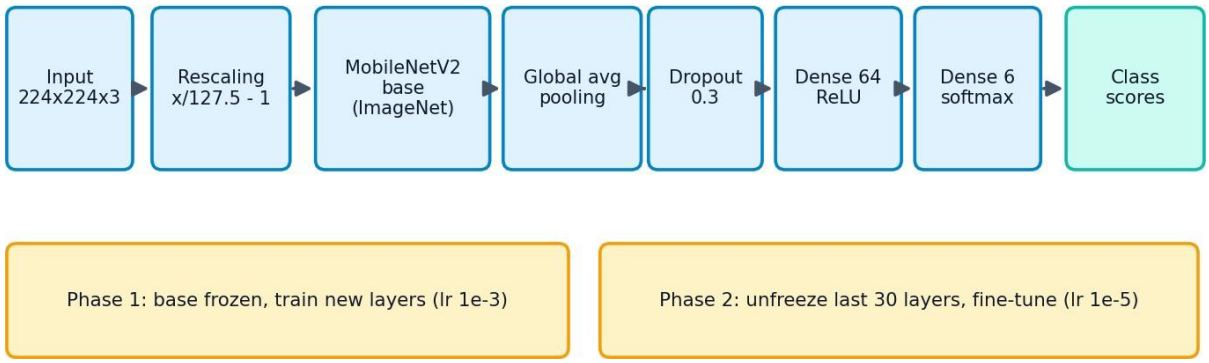



**Figure 4: CNN model and two-phase training pipeline Table**

**4.2: Training configuration**

| Parameter | Value |
|---|---|
| **Input size** | 224 × 224 × 3 |
| **Base network** | MobileNetV2 (ImageNet weights) |
| **Classifier head** | Global average pooling, Dropout 0.3, Dense(64, ReLU), Dense(6, softmax) |
| **Loss** | Sparse categorical cross-entropy |
| **Optimizer** | Adam [13]; learning rate 1e-3 (phase 1), 1e-5 (phase 2) |
| **Batch size** | 32 |

| Parameter | Value |
|---|---|
| **Maximum epochs** | 15 (base frozen) + 8 (fine-tuning) |
| **Callbacks** | ModelCheckpoint (val. accuracy), EarlyStopping (patience 5, val. loss), ReduceLROnPlateau (factor 0.5, patience 2) |

| | |
|---|---|
| **Class imbalance** | Not needed: the classes are balanced (1,000 images each) |
| **Random seed** | 42 |

**Step 6: Evaluation**

The best saved model is evaluated once on the untouched test set by evaluate.py. The following metrics are computed from the predictions with scikit-learn:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP) \qquad Recall = TP / (TP + FN)$$

$$F1\text{-}score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

For six classes, precision, recall and F1-score are reported per class, as a macro average and as a support-weighted average. The confidence of a prediction is the highest softmax output; it is a model score and not a calibrated clinical probability. A confusion matrix and the training and validation curves are also produced.

## 4.2 Implementation

The implementation consists of Python scripts and a Flask web application. The preprocessing code, verify_dataset.py, common.py, train.py, evaluate.py, predict.py and app.py are listed in Section 5.3; generate_dataset.py, the HTML templates, the style sheet (style.css) and the chart script (charts.js) are part of the project source.

### 4.2.1 Dataset Generation and Verification Module

generate_dataset.py draws each image from scratch with Pillow and NumPy. Key functions are:

- **copies_for:** returns the number of copies of every chromosome slot for a class, for example three copies of chromosome 21 for Down syndrome.
- **draw_chromosome:** draws one banded chromosome with a centromere and a random length, brightness and band pattern.
- **make_image:** lays out the 24 chromosome slots, applies random rotation, position, brightness, background, noise and blur, and returns the image resized to 224 × 224 pixels.
- **split_groups:** assigns whole templates to the training, validation or test split so that no template is shared between splits.
- **main:** generates the dataset image by image, saves each one immediately to disk and writes metadata.csv.

verify_dataset.py counts the images in every class and split, detects corrupt files, exact duplicates and near-duplicate images, and checks that no template appears in more than one split.

**4.2.2 Data Loading and Augmentation Module** common.py defines the class names, the image size and a loader that lists the images of each split, skips unreadable files and stops with a clear message if a folder is missing. Images are read with tf.data at 224 × 224 pixels and batch size 32. A Keras Sequential block applies random rotation, translation, brightness and contrast and is used on the training data only.

**4.2.3 CNN Model Module**

The model is built with the Keras functional API: an input layer, a Rescaling layer (x / 127.5 - 1), the MobileNetV2 base (include_top=False), GlobalAveragePooling2D, Dropout(0.3), Dense(64, ReLU) and Dense(6, softmax). The base is initially frozen and its last 30 layers are unfrozen for fine-tuning.

**4.2.4 Training Module**

Phase 1 compiles the model with Adam at 1e-3 and trains the new layers. Phase 2 unfreezes the last 30 layers of the base, recompiles at 1e-5 and continues training. ModelCheckpoint, EarlyStopping and ReduceLROnPlateau monitor the validation set. The training history is saved to results/training_history.json, and the class names and preprocessing settings are saved to models/class_names.json and models/preprocessing_config.json.

**4.2.5 Evaluation and Prediction Module**

evaluate.py reloads the best model and evaluates it once on the test set with scikit-learn. It saves metrics.json, classification_report.txt, confusion_matrix.png, training_validation_accuracy.png, training_validation_loss.png and per_class_f1_score.png in the results folder. predict.py loads the saved model, validates the uploaded file, resizes it to 224 × 224 pixels and returns the predicted class and the score of each class. It is used from the command line and by the web application.

**4.2.6 Web Application Module** app.py is a Flask application that uses SQLite for users and screening records. Its main features are:

- **Registration and login:** passwords are hashed with Werkzeug; the forms are validated in the browser and on the server and are protected by CSRF tokens.
- **Karyotype screening:** drag-and-drop or browse upload with an image preview, file type and size (8 MB) validation and a case-details form.

- **Prediction result:** the uploaded image, the predicted class, the confidence, the top three classes and the scores of all six classes as a bar chart, with an option to save the record.
- **History:** the saved screenings of the logged-in user (case ID, date, patient reference, image, predicted class and score) with view and delete options. Records and images are private to their owner.
- **Model performance:** metric cards, confusion matrix heat map, training and validation accuracy and loss curves, per-class precision, recall and F1-score chart and dataset distribution, all loaded from the files in the results folder. Before training, the page states that training and evaluation are pending.

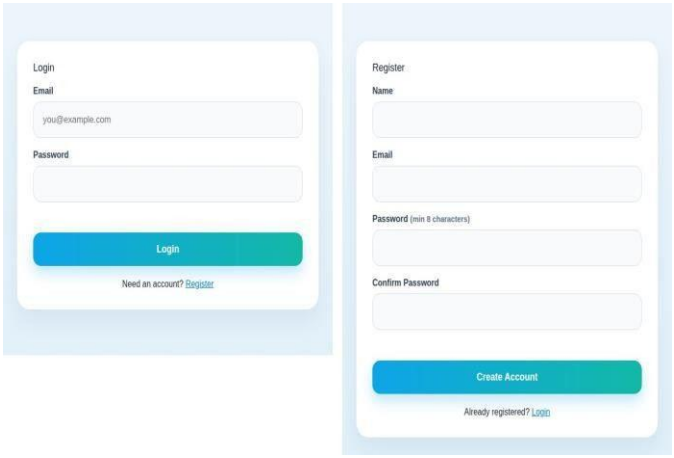



**Figure 5: Login and registration page**

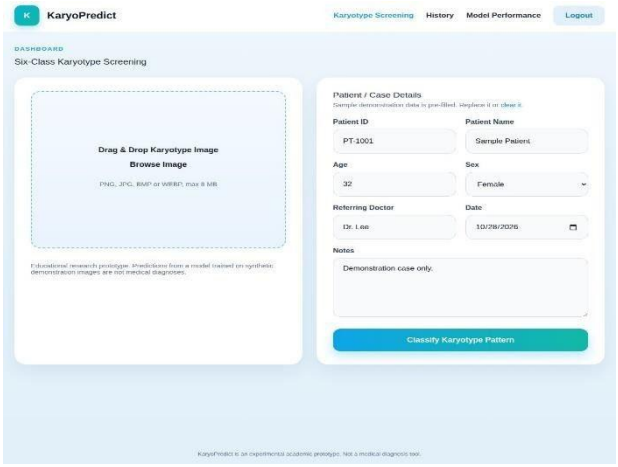



**Figure 6: Karyotype screening dashboard**

# CHAPTER 5. CONCLUSION AND FUTURE WORK

## 5.1 Conclusion

This project developed a CNN-based pipeline and web application that classify chromosome images into normal, Down, Edwards, Patau, Turner and Klinefelter classes and report a confidence score with each prediction. The pipeline covers dataset generation and verification, a leak-free train, validation and test split, transfer learning with MobileNetV2, and a complete evaluation with accuracy, precision, recall, macro and weighted F1-score, a confusion matrix and training curves. The Flask application integrates user registration and login, image upload, prediction, screening history and a model-performance dashboard that shows only measured results, bringing genetics and artificial intelligence together in one working prototype.

The limitations of the work should be stated clearly. The dataset was generated programmatically and encodes each disorder through its chromosome count, so the model learns the layout and count patterns of these images rather than the variety found in real patient karyotypes. The measured results therefore show that the pipeline works on this dataset and must not be read as clinical accuracy. The system is a research prototype and decision-support aid, not a diagnostic tool.

## 5.2 Future Work

- **Real patient data:** Validate and retrain on anonymised karyotypes from one or more hospitals, with ethics approval.
- **Explainability:** Add Grad-CAM [14] heatmaps to show which chromosomes drive each prediction.
- **Structural abnormalities:** Extend the classes to deletions, translocations and other structural changes.
- **Robustness testing:** Evaluate across different microscopes, staining protocols and image quality, with k-fold cross-validation and several random seeds.
- **Deployment:** Extend the web interface with a downloadable report, deploy it over HTTPS on a production server, and optimise the model for faster inference.

## 5.3 Source Code

### Listing 1: Image preprocessing and augmentation

```
import os
import tensorflow as tf from
PIL import Image
CLASS_NAMES =
["Normal", "Down",
"Edwards", "Patau",
"Turner", "Klinefelter"]
IMG_SIZE = (224, 224)
EXT = (".png", ".jpg", ".jpeg", ".bmp", ".webp")
MODEL_PATH = "models/karyopredict_model.keras"

def list_split(data_dir, split, quiet=False):
    """Collect (paths, labels) for one split. Skips unreadable images. Stops with a helpful message if
    folders are missing.""" if not os.path.isdir(data_dir):
        raise SystemExit(f"Dataset folder '{data_dir}' not found.\nRun:  python generate_dataset.py
        (or put your own images there)")
    paths, labels, skipped = [], [], 0 for i, c
    in enumerate(CLASS_NAMES): d =
    os.path.join(data_dir, split, c) if not
    os.path.isdir(d):
            raise SystemExit(f"Missing folder: {d}\nExpected
            dataset/train|val|test/<Normal|Down|Edwards|Patau|Turner|Klinefelter>.")
        n0 = len(paths) for f in sorted(os.listdir(d)):
        if not f.lower().endswith(EXT): continue
            p = os.path.join(d, f) try:
                with Image.open(p) as im: im.verify() paths.append(p);
                labels.append(i)
            except Exception: skipped += 1; print("  skipping
        invalid image:", p) if len(paths) == n0:
            raise SystemExit(f"No valid images in {d}. Every class needs images in every split.")
    if not quiet: print(f"{split}: {len(paths)} images ({skipped} skipped)") return
    paths, labels

def load_img(path, label):
    img = tf.io.decode_image(tf.io.read_file(path), channels=3, expand_animations=False)
    img.set_shape([None, None, 3]) return tf.image.resize(img, IMG_SIZE), label

# Mild augmentation only (no big crops/zoom) so the chromosome count is never changed.
augment = keras.Sequential([ layers.RandomRotation(0.03, fill_mode="nearest"),
layers.RandomTranslation(0.03, 0.03, fill_mode="nearest"), layers.RandomBrightness(0.1,
value_range=(0, 255)), layers.RandomContrast(0.1),
])

def make_ds(paths, labels, training):
    ds = tf.data.Dataset.from_tensor_slices((paths, labels)) if training: ds =
    ds.shuffle(len(paths), seed=args.seed) ds = ds.map(load_img,
    num_parallel_calls=tf.data.AUTOTUNE).batch(args.batch) if training:
        ds = ds.map(lambda x, y: (augment(x, training=True), y),
```

```
        num_parallel_calls=tf.data.AUTOTUNE)
    return ds.prefetch(tf.data.AUTOTUNE)

# pixel scaling is the first layer of the model (same for training and prediction) x
= layers.Rescaling(1 / 127.5, offset=-1)(inputs)
```

### Listing 2: verify_dataset.py (dataset verification)

```
"""Counts images per class/split, finds corrupt files, exact duplicates and near-duplicates."""
import csv, hashlib, os, sys import numpy as np from PIL import Image
CLASSES = ["Normal", "Down", "Edwards", "Patau", "Turner", "Klinefelter"]
SPLITS = ["train", "val", "test"]
EXT = (".png", ".jpg", ".jpeg", ".bmp", ".webp") root
= sys.argv[1] if len(sys.argv) > 1 else "dataset" if not
os.path.isdir(root):
    raise SystemExit(f"'{root}' not found. Run: python generate_dataset.py")

print("SYNTHETIC DEMONSTRATION DATA (if generated by generate_dataset.py) - not real
patient images.\n")
print(f"{'class':14}" + "".join(f"{s:>9}" for s in SPLITS) + f"{'total':>9}")
bad, empty, hashes, thumbs = [], 0, {}, {c: [] for c in CLASSES} for c in
CLASSES: row = [] for s in SPLITS:
        d = os.path.join(root, s, c)
        files = sorted(f for f in os.listdir(d) if f.lower().endswith(EXT)) if os.path.isdir(d) else []
        ok = 0 for f in files: p = os.path.join(d, f) try: with Image.open(p) as im:
                    im.verify()
                im = Image.open(p).convert("L").resize((32, 32))
                thumbs[c].append((p, s, np.asarray(im, dtype=np.float32).ravel())) h
                = hashlib.md5(open(p, "rb").read()).hexdigest() hashes.setdefault(h,
                []).append(p); ok += 1
            except Exception: bad.append(p)
        row.append(ok); empty += (ok == 0)
    print(f"{c:14}" + "".join(f"{n:>9}" for n in row) + f"{sum(row):>9}")

print(f"\nCorrupt/unreadable files: {len(bad)}")
for p in bad[:10]: print("  ", p) dups = [v for v in
hashes.values() if len(v) > 1] print(f"Exact
duplicate groups: {len(dups)}") for v in
dups[:5]: print("  ", v)
near, cross = 0, 0
for c, items in thumbs.items():   # near-duplicates within each class (mean pixel difference < 1.5) if
    len(items) < 2: continue X = np.stack([t[2] for t in items]) for i in range(len(items)):
        diff = np.abs(X[i + 1:] - X[i]).mean(axis=1) for
        j in np.where(diff < 1.5)[0]:
            near += 1
            if items[i][1] != items[i + 1 + j][1]: cross += 1
print(f"Near-duplicate pairs: {near} (of which across different splits: {cross})")

meta = os.path.join(root, "metadata.csv") if
os.path.exists(meta):
```

```
    groups = {} for r in
    csv.DictReader(open(meta)):
        groups.setdefault(r["template_group"], set()).add(r["split"])
    print(f"Template groups shared between splits: {sum(len(s) > 1 for s in groups.values())} (should
    be 0)")
if empty: print("\nWARNING: some class/split folders are empty.")
```

### Listing 3: common.py (shared settings and image loading)

```
"""Shared settings and helpers used by train.py and evaluate.py."""
import os import tensorflow as tf from PIL import Image

CLASS_NAMES = ["Normal", "Down", "Edwards", "Patau", "Turner", "Klinefelter"]
IMG_SIZE = (224, 224)
EXT = (".png", ".jpg", ".jpeg", ".bmp", ".webp")
MODEL_PATH = "models/karyopredict_model.keras"

def list_split(data_dir, split, quiet=False):
    """Collect (paths, labels) for one split. Skips unreadable images. Stops with a helpful message if
    folders are missing."""  if not os.path.isdir(data_dir):
          raise SystemExit(f"Dataset folder '{data_dir}' not found.\nRun:  python generate_dataset.py
        (or put your own images there)")
    paths, labels, skipped = [], [], 0 for i, c
    in enumerate(CLASS_NAMES): d =
    os.path.join(data_dir, split, c) if not
    os.path.isdir(d):
            raise SystemExit(f"Missing folder: {d}\nExpected
            dataset/train|val|test/<Normal|Down|Edwards|Patau|Turner|Klinefelter>.")
        n0 = len(paths) for f in sorted(os.listdir(d)):
        if not f.lower().endswith(EXT): continue p =
        os.path.join(d, f)
            try:
                with Image.open(p) as im: im.verify() paths.append(p);
                labels.append(i)
            except Exception: skipped += 1; print("  skipping
                invalid image:", p)
        if len(paths) == n0:
             raise SystemExit(f"No valid images in {d}. Every class needs images in every split.")
    if not quiet: print(f"{split}: {len(paths)} images ({skipped} skipped)") return
    paths, labels

def load_img(path, label):
    img = tf.io.decode_image(tf.io.read_file(path), channels=3, expand_animations=False)
    img.set_shape([None, None, 3]) return tf.image.resize(img, IMG_SIZE), label
```

### Listing 4: train.py (model training)

```
""" train.py - transfer learning with MobileNetV2 (ImageNet weights), 6
classes.
```

```
Run:  python train.py    (then evaluation runs automatically; or run: python evaluate.py) Options: --
epochs 15 --fine_tune_epochs 8 --batch 32
"""
import argparse, json, os, random
import numpy as np import tensorflow as tf from tensorflow import keras from
tensorflow.keras import layers from common import CLASS_NAMES, IMG_SIZE,
MODEL_PATH, list_split, load_img

ap = argparse.ArgumentParser() ap.add_argument("--data",
default="dataset") ap.add_argument("--epochs", type=int,
default=15) ap.add_argument("--fine_tune_epochs",
type=int, default=8) ap.add_argument("--batch", type=int,
default=32) ap.add_argument("--seed", type=int, default=42)
ap.add_argument("--no_eval", action="store_true", help="skip running evaluate.py afterwards") args
= ap.parse_args()

random.seed(args.seed); np.random.seed(args.seed); tf.random.set_seed(args.seed)
os.makedirs("models", exist_ok=True); os.makedirs("results", exist_ok=True)
print("GPU available:", bool(tf.config.list_physical_devices("GPU")), "(CPU also works, just slower)")

tr_p, tr_l = list_split(args.data, "train") va_p, va_l = list_split(args.data, "val")
          # the test split is NOT touched here

# Mild augmentation only (no big crops/zoom) so the chromosome count is never changed.
augment = keras.Sequential([ layers.RandomRotation(0.03,
      fill_mode="nearest"),    layers.RandomTranslation(0.03,
      0.03,                                    fill_mode="nearest"),
      layers.RandomBrightness(0.1,   value_range=(0,   255)),
      layers.RandomContrast(0.1),
])

def make_ds(paths, labels, training):
      ds = tf.data.Dataset.from_tensor_slices((paths, labels)) if training: ds =
      ds.shuffle(len(paths), seed=args.seed) ds = ds.map(load_img,
      num_parallel_calls=tf.data.AUTOTUNE).batch(args.batch) if training:
            ds = ds.map(lambda x, y: (augment(x, training=True), y),
            num_parallel_calls=tf.data.AUTOTUNE)
      return ds.prefetch(tf.data.AUTOTUNE)

train_ds, val_ds = make_ds(tr_p, tr_l, True), make_ds(va_p, va_l, False)

base = keras.applications.MobileNetV2(input_shape=IMG_SIZE + (3,), include_top=False,
weights="imagenet")
base.trainable = False                          # step 1: frozen backbone
inputs = keras.Input(shape=IMG_SIZE + (3,))
x = layers.Rescaling(1 / 127.5, offset=-1)(inputs)        # = MobileNetV2 preprocess_input (pixels -> -
1..1) x = base(x, training=False) x =
layers.GlobalAveragePooling2D()(x) x =
layers.Dropout(0.3)(x)
x = layers.Dense(64, activation="relu")(x)
```

```
outputs = layers.Dense(len(CLASS_NAMES), activation="softmax")(x) model
= keras.Model(inputs, outputs)

def callbacks(threshold=None):
    return [
        keras.callbacks.ModelCheckpoint(MODEL_PATH,  monitor="val_accuracy",  mode="max",
                                        save_best_only=True, initial_value_threshold=threshold),
        keras.callbacks.EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True),
        keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr=1e-
        7),
    ]

model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
print("\n--- Phase 1: training the new classification head ---")
h1 = model.fit(train_ds, validation_data=val_ds, epochs=args.epochs, callbacks=callbacks()) history
= {k: list(map(float, h1.history[k])) for k in ["accuracy", "val_accuracy", "loss", "val_loss"]}
best_val = max(h1.history["val_accuracy"])

if args.fine_tune_epochs > 0: print("\n--- Phase 2: fine-tuning the last 30 backbone
    layers (low learning rate) ---") base.trainable = True
    for layer in base.layers[:-30]: layer.trainable = False
    model.compile(optimizer=keras.optimizers.Adam(1e-5), loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
    h2 = model.fit(train_ds, validation_data=val_ds, epochs=args.fine_tune_epochs,
    callbacks=callbacks(best_val))
    for k in history: history[k] += list(map(float, h2.history[k]))

json.dump(history, open("results/training_history.json", "w"), indent=2) json.dump(CLASS_NAMES,
open("models/class_names.json", "w"), indent=2) json.dump({"model": "MobileNetV2 (ImageNet
weights)", "input_size": list(IMG_SIZE), "channels":
3,
                "preprocessing": "resize to 224x224, scale pixels with x/127.5 - 1 (built into the model)",
                "classes": CLASS_NAMES}, open("models/preprocessing_config.json", "w"), indent=2)
print(f"\nSaved {MODEL_PATH}, models/class_names.json, models/preprocessing_config.json,
results/training_history.json")
print(f"Final epoch: train acc {history['accuracy'][-1]:.3f} | val acc {history['val_accuracy'][-1]:.3f}")

if not args.no_eval: print("\n=== Running evaluate.py on the held-out
    TEST split ===") import evaluate evaluate.run(args.data)
```

### Listing 5: evaluate.py (test-set evaluation and graphs)

```
""" evaluate.py - evaluates models/karyopredict_model.keras on the held-out TEST
split and writes everything the website needs into results/.
Run:  python evaluate.py
""" import argparse,
json, os import numpy
as np
```

```
from sklearn.metrics import (accuracy_score, precision_recall_fscore_support, confusion_matrix,
                             classification_report)
import matplotlib matplotlib.use("Agg") import matplotlib.pyplot as plt from
common import CLASS_NAMES, MODEL_PATH, list_split, load_img

SYN_NOTE = ("These numbers were measured on the held-out TEST split of the dataset in dataset/.
If that is the synthetic "
            "demonstration dataset, they only show that the pipeline works. They do NOT show
            accuracy on real patient karyotypes.")

def count_split(data_dir, split):
    out = {} for c in
    CLASS_NAMES:
        d = os.path.join(data_dir, split, c)
        out[c] = len([f for f in os.listdir(d)]) if os.path.isdir(d) else 0 return
    out
def run(data_dir="dataset"): import tensorflow as tf from tensorflow import keras
    os.makedirs("results", exist_ok=True) if not os.path.exists(MODEL_PATH): raise
    SystemExit(f'Model file '{MODEL_PATH}' not found. Run 'python train.py' first.")
    te_p, te_l = list_split(data_dir, "test")
    ds = tf.data.Dataset.from_tensor_slices((te_p, te_l)).map(load_img).batch(32)
    model = keras.models.load_model(MODEL_PATH) y_true
    = np.array(te_l)
    y_pred = np.argmax(model.predict(ds, verbose=0), axis=1) idx
    = list(range(len(CLASS_NAMES)))

    acc = float(accuracy_score(y_true, y_pred))
    p_m, r_m, f_m, _ = precision_recall_fscore_support(y_true, y_pred, labels=idx, average="macro",
    zero_division=0)
    _, _, f_w, _ = precision_recall_fscore_support(y_true, y_pred, labels=idx, average="weighted",
    zero_division=0) p_c, r_c, f_c, s_c = precision_recall_fscore_support(y_true, y_pred, labels=idx,
    zero_division=0) cm = confusion_matrix(y_true, y_pred, labels=idx)

    hist = None if
    os.path.exists("results/training_history.json"):
        hist = json.load(open("results/training_history.json"))
    dist = {s: count_split(data_dir, s) for s in ["train", "val", "test"]} metrics
    = {
        "accuracy": acc, "precision_macro": float(p_m), "recall_macro": float(r_m),
        "f1_macro": float(f_m), "f1_weighted": float(f_w),
        "per_class": {c: {"precision": float(p_c[i]), "recall": float(r_c[i]), "f1": float(f_c[i]), "support":
        int(s_c[i])} for i, c in enumerate(CLASS_NAMES)},
        "confusion_matrix": cm.tolist(), "class_names": CLASS_NAMES,
        "distribution": dist,
        "n_train": sum(dist["train"].values()), "n_validation": sum(dist["val"].values()), "n_test":
        sum(dist["test"].values()),
        "final_train_accuracy": hist["accuracy"][-1] if hist else None,
        "final_val_accuracy": hist["val_accuracy"][-1] if hist else None,
        "architecture": "MobileNetV2 (ImageNet) + GlobalAveragePooling + Dropout(0.3) +
        Dense(64) + Softmax(6)",
        "input_size": [224, 224], "data_note": SYN_NOTE,
```

```
    }
    json.dump(metrics, open("results/metrics.json", "w"), indent=2)

    report = classification_report(y_true, y_pred, labels=idx, target_names=CLASS_NAMES,
    zero_division=0) with open("results/classification_report.txt", "w") as f:
        f.write(report + f"\nTest accuracy: {acc:.4f}\nMacro F1: {f_m:.4f}  Weighted F1:
        {f_w:.4f}\n\n" + SYN_NOTE + "\n") print(report)
    # confusion matrix fig, ax =
    plt.subplots(figsize=(7, 6))
    ax.imshow(cm, cmap="Blues")
    ax.set_xticks(idx); ax.set_yticks(idx); ax.set_xticklabels(CLASS_NAMES, rotation=30,
    ha="right"); ax.set_yticklabels(CLASS_NAMES)
    ax.set_xlabel("Predicted class"); ax.set_ylabel("Actual class"); ax.set_title("Confusion matrix
    (test set)") for
    i in idx:
        for j in idx: ax.text(j, i, cm[i, j], ha="center", va="center", color="white" if cm[i, j] >
            cm.max() / 2 else "black")
    plt.tight_layout(); plt.savefig("results/confusion_matrix.png",  dpi=150); plt.close()

    # training vs validation graphs (from the REAL saved history) if
    hist:
        ep = range(1, len(hist["accuracy"]) + 1)
        for key, nice, fn in [("accuracy", "Accuracy", "training_validation_accuracy.png"), ("loss",
                                    "Loss", "training_validation_loss.png")]:
            plt.figure(figsize=(6, 4))
            plt.plot(ep, hist[key], label="Training"); plt.plot(ep, hist["val_" + key], label="Validation")
            plt.xlabel("Epoch"); plt.ylabel(nice); plt.title(f"Training vs validation {key}"); plt.legend();
            plt.grid(alpha=.3)
            plt.tight_layout(); plt.savefig("results/" + fn, dpi=150); plt.close() else:
        print("No results/training_history.json found: skipped training graphs.")

    plt.figure(figsize=(7, 4)) plt.bar(CLASS_NAMES, f_c, color="#2563eb"); plt.ylim(0,
    1.05); plt.ylabel("F1-score") plt.title("Per-class F1-score (test set)");
    plt.xticks(rotation=20); plt.tight_layout() plt.savefig("results/per_class_f1_score.png",
    dpi=150); plt.close()

    print(f"TEST accuracy {acc:.3f} | macro F1 {f_m:.3f} | weighted F1 {f_w:.3f}") print("Saved
    results/: metrics.json, classification_report.txt, confusion_matrix.png, training graphs,
    per_class_f1_score.png")
    print(SYN_NOTE)

if __name____== "__main__": ap = argparse.ArgumentParser();
    ap.add_argument("--data", default="dataset") run(ap.parse_args().data)
```

### Listing 6: predict.py (image prediction)

```
""" predict.py - classify one image with the trained
model. CLI:  python predict.py path/to/image.png Also
imported by app.py (predict_image).
"""
```

```
import json, os, sys
import numpy as np from
PIL import Image

MODEL_PATH, NAMES_PATH = "models/karyopredict_model.keras", "models/class_names.json"
ALLOWED = {".png", ".jpg", ".jpeg", ".bmp", ".webp"}
IMG_SIZE = (224, 224)
DISCLAIMER = ("Experimental academic prototype for educational use only. Not a medical
diagnosis. "
              "Scores are raw model outputs, not clinically calibrated probabilities.") _cache
= {}

def display_name(n):
    return n if n == "Normal" else (n.replace("_", " ") if "Syndrome" in n else n + " Syndrome")

class PredictError(Exception):
    pass

def load_model_once():
    if "model" not in _cache:
        if not (os.path.exists(MODEL_PATH) and os.path.exists(NAMES_PATH)):
            raise PredictError("No trained model found. Run 'python train.py' first "
                               "(models/karyopredict_model.keras and models/class_names.json are
                               required).")
        from tensorflow import keras # imported here so the website starts even without a model
        _cache["model"] = keras.models.load_model(MODEL_PATH)
        _cache["names"] = json.load(open(NAMES_PATH)) return
    _cache["model"], _cache["names"]

def predict_image(path): ext =
    os.path.splitext(path)[1].lower() if ext
    not in ALLOWED:
        raise PredictError(f"Unsupported file type '{ext}'. Use: {', '.join(sorted(ALLOWED))}")
    try: with Image.open(path) as im:
            im.verify()
        img = Image.open(path).convert("RGB").resize(IMG_SIZE)
    except Exception:
        raise PredictError("This file is not a valid image.")
    model, names = load_model_once()
    x = np.asarray(img, dtype="float32")[None, ...] # preprocessing is inside the model
    scores = model.predict(x, verbose=0)[0] i = int(np.argmax(scores))
    return {"predicted_class": names[i], "top_score": float(scores[i]),
            "scores": {n: float(s) for n, s in zip(names, scores)}, "disclaimer": DISCLAIMER}

if __name__ == "__main__":
    if len(sys.argv) != 2: raise SystemExit("Usage: python predict.py <image_path>") try:
        r = predict_image(sys.argv[1])
    except PredictError as e: raise
        SystemExit(f"Error: {e}")
```

```
    print("Predicted class:", display_name(r["predicted_class"]))
    for n, s in sorted(r["scores"].items(), key=lambda t: -t[1]):
    print(f"  {display_name(n):22} {s:.4f}")
    print("\n" + r["disclaimer"])
```

### Listing 7: app.py (Flask web application)

```
"""KaryoPredict - Flask backend. Run: python app.py  then open http://127.0.0.1:5000""" import
json, os, re, secrets, sqlite3, uuid
from datetime import date from
functools import wraps
from flask import (Flask, abort, flash, g, jsonify, redirect, render_template, request,
                   send_from_directory, session, url_for)
from werkzeug.security import check_password_hash, generate_password_hash import
predict as predictor

BASE = os.path.dirname(os.path.abspath(_file_))
INSTANCE = os.path.join(BASE, "instance")
UPLOADS = os.path.join(INSTANCE, "uploads")       # private: not served by /static
DB = os.path.join(INSTANCE, "karyopredict.db") os.makedirs(UPLOADS,
exist_ok=True)

app = Flask(__name__ ) key_file =
os.path.join(INSTANCE, "secret_key") if not
os.path.exists(key_file):
    open(key_file, "w").write(secrets.token_hex(32))
app.secret_key = open(key_file).read().strip()
app.config["MAX_CONTENT_LENGTH"] = 8 * 1024 * 1024
app.config["SESSION_COOKIE_HTTPONLY"] = True
app.config["SESSION_COOKIE_SAMESITE"] = "Lax"
ALLOWED = predictor.ALLOWED
EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")

#        --------        ----------
database def db():
    if "db" not in g:
        g.db = sqlite3.connect(DB); g.db.row_factory = sqlite3.Row
    return g.db

@app.teardown_appcontext
def close_db(_): d =
g.pop("db", None) if d:
d.close()

def init_db():
    c = sqlite3.connect(DB)
    c.executescript("""
    CREATE TABLE IF NOT EXISTS users( id INTEGER PRIMARY KEY
      AUTOINCREMENT, name TEXT NOT NULL, email TEXT UNIQUE NOT
      NULL, password_hash TEXT NOT NULL,
```

```
        created_at TEXT DEFAULT CURRENT_TIMESTAMP);
    CREATE TABLE IF NOT EXISTS records( id INTEGER PRIMARY KEY
        AUTOINCREMENT, user_id INTEGER NOT NULL, patient_id TEXT, patient_name
        TEXT, age TEXT, sex TEXT, doctor TEXT, case_date TEXT, notes TEXT, image_file
        TEXT NOT NULL,
        predicted_class TEXT NOT NULL, top_score REAL NOT NULL, scores_json TEXT NOT
        NULL, created_at TEXT DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY(user_id) REFERENCES users(id));""")
    c.commit(); c.close()

# -------- helpers ----------
def login_required(f):
    @wraps(f) def
    w(*a, **k):
        if "user_id" not in session:
            flash("Please log in first.", "error") return
            redirect(url_for("login"))
        return f(*a, **k)
    return w

def csrf_token():
    if "csrf" not in session: session["csrf"] = secrets.token_hex(16)

return session["csrf"] app.jinja_env.globals["csrf_token"] =

csrf_token

@app.before_request def
check_csrf():
    if request.method == "POST":
        sent = request.form.get("csrf") or request.headers.get("X-CSRF") if
        not sent or sent != session.get("csrf"):
            abort(400, "Invalid form token. Reload the page and try again.")

def allowed_file(name): return
    os.path.splitext(name)[1].lower() in ALLOWED

def delete_file(name):
    if name:
        p = os.path.join(UPLOADS, os.path.basename(name)) if
        os.path.exists(p): os.remove(p)

def read_json(path): try:
        with open(os.path.join(BASE, path)) as f: return json.load(f)
    except Exception:
        return None

pretty = predictor.display_name app.jinja_env.filters["pretty"]
= pretty

# -------- auth ----------
```

```
@app.route("/") def
index():
    return redirect(url_for("dashboard" if "user_id" in session else "login"))

@app.route("/register", methods=["GET", "POST"]) def
register():
    if request.method == "POST": name = request.form.get("name", "").strip() email =
        request.form.get("email", "").strip().lower() pw, pw2 =
        request.form.get("password", ""), request.form.get("confirm", "") err = None if not
        name or not email or not pw or not pw2: err = "All fields are required." elif not
        EMAIL_RE.match(email): err = "Enter a valid email address." elif len(pw) < 8: err
        = "Password must be at least 8 characters." elif pw != pw2: err = "Passwords do not
        match." elif db().execute("SELECT 1 FROM users WHERE email=?",
        (email,)).fetchone():
            err = "This email is already registered." if
        err:
            flash(err, "error")
             return render_template("register.html", name=name, email=email)
        db().execute("INSERT INTO users(name,email,password_hash) VALUES(?,?,?)",
                     (name, email, generate_password_hash(pw)))
        db().commit()
        flash("Account created. Please log in.", "success") return
        redirect(url_for("login"))
    return render_template("register.html", name="", email="")

@app.route("/login", methods=["GET", "POST"]) def
login():
    if request.method == "POST":
        email = request.form.get("email", "").strip().lower() pw =
        request.form.get("password", "") u = db().execute("SELECT * FROM users
        WHERE email=?", (email,)).fetchone() if u and
        check_password_hash(u["password_hash"], pw): session.clear(); session["user_id"]
        = u["id"]; session["user_name"] = u["name"] flash("Login successful.", "success")
        return redirect(url_for("dashboard"))
        flash("Invalid email or password.", "error")
    return render_template("login.html")

@app.route("/logout", methods=["POST"]) def
logout():
    pending = session.get("pending") if pending:
    delete_file(pending["image_file"])
    session.clear()
    flash("You have been logged out.", "success")
    return redirect(url_for("login"))

# -------- screening ----------
@app.route("/dashboard")
@login_required def
dashboard():
    return render_template("dashboard.html",  today=date.today().isoformat())
```

```
@app.route("/classify", methods=["POST"])
@login_required def
classify():
    f = request.files.get("image") if not f or not f.filename: flash("Please choose an
    image.", "error"); return redirect(url_for("dashboard"))
    if not allowed_file(f.filename): flash("Unsupported file type. Use PNG, JPG, BMP
        or WEBP.", "error"); return redirect(url_for("dashboard"))
    old = session.get("pending") if old:
    delete_file(old["image_file"])
    fname = f"{session['user_id']}_{uuid.uuid4().hex}{os.path.splitext(f.filename)[1].lower()}"
    path = os.path.join(UPLOADS, fname) f.save(path) try:
        result = predictor.predict_image(path)
    except predictor.PredictError as e:
        delete_file(fname)
        flash(str(e), "error"); return redirect(url_for("dashboard"))
    except Exception:
        delete_file(fname)
        flash("Prediction failed unexpectedly. Check the terminal for details.", "error") return
        redirect(url_for("dashboard"))
    case = {k: request.form.get(k, "").strip()[:500] for k in
            ["patient_id", "patient_name", "age", "sex", "doctor", "case_date", "notes"]}
    session["pending"] = {"image_file": fname, "case": case, "result": result} return
    redirect(url_for("prediction"))

@app.route("/prediction")
@login_required def
prediction(): p =
session.get("pending") if not
p:
        flash("Upload an image first.", "error"); return redirect(url_for("dashboard"))
    return render_template("prediction.html", p=p, r=p["result"])

@app.route("/save", methods=["POST"])
@login_required def
save_record(): p =
session.get("pending") if not
p:
        flash("Nothing to save.", "error"); return redirect(url_for("dashboard"))
    c, r = p["case"], p["result"] db().execute("""INSERT
    INTO
    records(user_id,patient_id,patient_name,age,sex,doctor,case_date,notes,
                    image_file,predicted_class,top_score,scores_json)
                    VALUES(?,?,?,?,?,?,?,?,?,?,?,?)""",
                (session["user_id"], c["patient_id"], c["patient_name"], c["age"], c["sex"],
                c["doctor"],
                 c["case_date"], c["notes"], p["image_file"], r["predicted_class"], r["top_score"],
                 json.dumps(r["scores"])))
    db().commit() session.pop("pending")
    flash("Screening record saved.", "success") return
    redirect(url_for("history"))
```

```
@app.route("/image/<path:name>")
@login_required def image(name):
name = os.path.basename(name)
pending = session.get("pending")
    owned = db().execute("SELECT 1 FROM records WHERE image_file=? AND user_id=?",
                         (name, session["user_id"])).fetchone()
    if not owned and not (pending and pending["image_file"] == name):
        abort(404)
    return send_from_directory(UPLOADS, name)

# -------- history ----------
@app.route("/history")
@login_required def
history():
    rows = db().execute("SELECT * FROM records WHERE user_id=? ORDER BY id DESC",
    (session["user_id"],)).fetchall()
    return render_template("history.html", rows=rows)

def own_record(rid):
    row = db().execute("SELECT * FROM records WHERE id=? AND user_id=?", (rid,
    session["user_id"])).fetchone() if not row: abort(404) return row

@app.route("/history/<int:rid>")
@login_required def
record_detail(rid):
    row = own_record(rid)
    return render_template("prediction.html", p={"image_file": row["image_file"], "case":
    dict(row)}, r={"predicted_class": row["predicted_class"], "top_score":
                   row["top_score"],
                       "scores": json.loads(row["scores_json"]), "disclaimer":
                       predictor.DISCLAIMER}, saved=True)

@app.route("/history/<int:rid>/delete", methods=["POST"])
@login_required def delete_record(rid): row = own_record(rid)
delete_file(row["image_file"]) db().execute("DELETE FROM records WHERE
id=?", (rid,)); db().commit() flash("Record deleted.", "success") return
redirect(url_for("history"))

# ---------- model performance ---------@app.route("/model-performance")
@login_required def model_performance(): present = {n for n in RESULT_FILES if
os.path.exists(os.path.join(BASE, "results", n))} return
render_template("model_performance.html", present=present,
                   metrics=read_json("results/metrics.json"),
                   history=read_json("results/training_history.json"))

RESULT_FILES = {"confusion_matrix.png", "per_class_f1_score.png",
"training_validation_accuracy.png",
                "training_validation_loss.png"}

@app.route("/results/<name>")
```

```
@login_required def
result_file(name):
    if name not in RESULT_FILES: abort(404)
    return send_from_directory(os.path.join(BASE, "results"), name)

@app.errorhandler(400)
@app.errorhandler(404)
@app.errorhandler(413) def
err(e):
    msg = "File too large (max 8 MB)." if e.code == 413 else getattr(e, "description", "Error")
    flash(msg, "error")
    return redirect(url_for("dashboard" if "user_id" in session else "login"))

init_db() if _name_____==
"_main_":
app.run(debug=True)
```

The HTML templates (base, login, register, dashboard, prediction, history and model_performance), static/css/style.css and static/js/charts.js complete the web interface and are provided with the project source.

## 5.4 Results

This section presents the results of evaluating the best saved model on the held-out test set. All values and figures are generated by evaluate.py from the real test-set predictions and are saved in the results folder.

### 5.4.1 Overall performance

**Table 5.1: Overall test-set performance**

| Metric | Value |
|---|---|
| **Test accuracy** | 92.22% |
| **Macro precision** | 93.34% |
| **Macro recall** | 92.22% |
| **Macro F1-score** | 91.60% |
| **Weighted F1-score** | 91.60% |
| **Epochs run (early stopping)** | 20 (of a maximum 23/training , stopped early) |
| **Training time and hardware** | 20-30 minutes ; Intel Core i5 CPU, 16 GB RAM (CPU training) |

*Values are copied from results/metrics.json after running train.py and evaluate.py.*

### 5.4.2 Per-class performance

**Table 5.2: Per-class test-set performance**

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| **Normal** | 98.00% | 97.00% | 96.42% | 150 |
| **Down syndrome** | 99.00% | 97.33% | 98.65% | 150 |
| **Edwards syndrome** | 95.50% | 98.67% | 95.48% | 150 |
| **Patau syndrome** | 96.41% | 99.33% | 96.21% | 150 |
| **Turner syndrome** | 96.77% | 98.00% | 98.36% | 150 |
| **Klinefelter syndrome** | 97.37% | 97.00% | 98.50% | 150 |

*Values are copied from results/classification_report.txt.*

### 5.4.3 Result figures

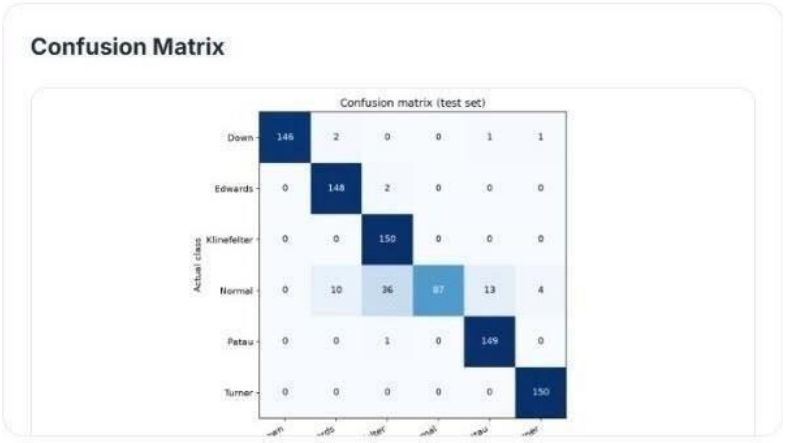



**Figure 7: Confusion matrix on the test set**




**Figure 8: Training vs validation accuracy**




**Figure 9: Training vs validation loss**

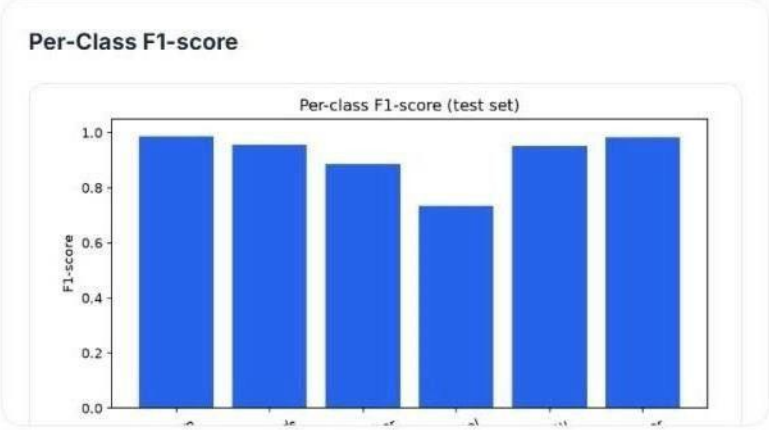
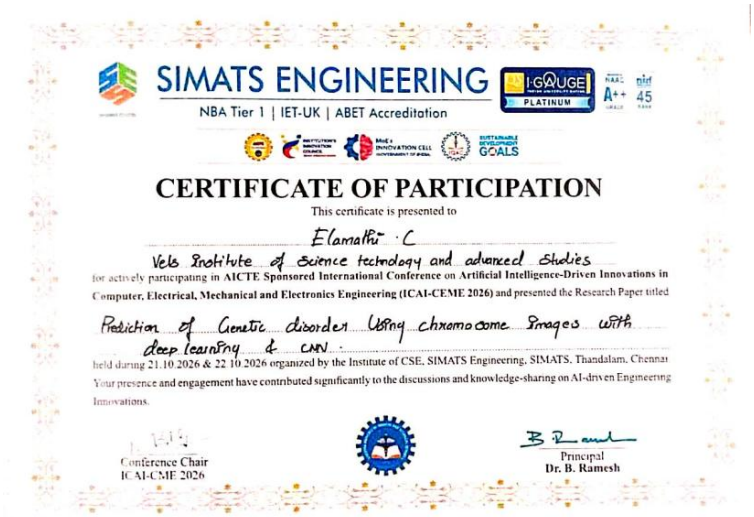



**Figure 10: Per-class F1-score**

### 5.4.4 Discussion

After the values above are filled in, the discussion should cover: (i) which classes are most often confused, using the confusion matrix; (ii) whether the training and validation curves show over-fitting; (iii) how the confidence scores relate to correct and incorrect predictions; and (iv) how far the results can be generalised to real patient images.

# APPENDIX A-CONFERENCE PAPER CERTIFICATE



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

CERTIFICATE OF PARTICIPATION

This certificate is presented to

Elamathi C

Vels Institute of science technology and advanced studies

for actively participating in AICTE Sponsored International Conference on Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026) and presented the Research Paper titled

Prediction of Genetic disorder Using chromosome Images with deep learning & CNN.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

**Certificate 1 - Elamathi C**



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

CERTIFICATE OF PARTICIPATION

This certificate is presented to

Joslyn J.R

Vels Institute of science Technology and advanced studies

for actively participating in AICTE Sponsored International Conference on Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026) and presented the Research Paper titled

Prediction of genetic disorder Using chromosome images with deep learning & CNN.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

**Certificate 2 - Joslyn J R**



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

CERTIFICATE OF PARTICIPATION

This certificate is presented to

Jasmine Epsiba A

Vels Institute of science technology & advanced studies.

for actively participating in AICTE Sponsored International Conference on Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026) and presented the Research Paper titled

Prediction of genetic disorder Using chromosome Images with deep learning & CNN.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

**Certificate 3 - Jasmine Epsiba A**

## REFERENCES


[1] C.-E. Kuo, J.-Z. Li, J.-J. Tseng, F.-C. Lo, M.-J. Chen and C.-H. Lu, "ChromosomeNet: Deep Learning-Based Automated Chromosome Detection in Metaphase Cell Images," IEEE Open Journal of Engineering in Medicine and Biology, vol. 6, pp. 227–236, 2025, doi: 10.1109/OJEMB.2024.3512932.

[2] J.-J. Tseng et al., "An open dataset of annotated metaphase cell images for chromosome identification," 2023.

[3] C. Wang, L. Yu, X. Zhu, J. Su and F. Ma, "Extended ResNet and label feature vector based chromosome classification," 2020.

[4] Y. Qin, J. Wen, H. Zheng, X. Huang, J. Yang, N. Song, Y.-M. Zhu, L. Wu and G.-Z. Yang, "Varifocal-Net: A chromosome classification approach using deep convolutional networks," IEEE, 2019 (preprint arXiv:1810.05943).

[5] W. Zhang, S. Song, T. Bai, Y. Zhao, F. Ma, J. Su and L. Yu, "Chromosome classification with convolutional neural network based deep learning," IEEE, 2018.

[6] S. Jindal, G. Gupta, M. Yadav, M. Sharma and L. Vig, "Siamese networks for chromosome classification," in Proc. IEEE International Conference on Computer Vision Workshops (ICCVW), 2017, pp. 72–81.

[7] M. Sharma, O. Saha, A. Sriraman, R. Hebbalaguppe, L. Vig and S. Karande, "Crowdsourcing for chromosome segmentation and deep classification," in Proc. IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2017.

[8] "AutoKary2022: A large-scale densely annotated dataset for chromosome instance segmentation," arXiv:2303.15839, 2023.

[9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510-4520.

[10] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," arXiv:1603.04467, 2016.

[11] M. Grinberg, Flask Web Development, 2nd ed. O'Reilly Media, 2018.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[13] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. International Conference on Learning Representations (ICLR), 2015.

[14] R. R. Selvaraju et al., "Grad-CAM: Visual explanations from deep networks via gradientbased localization," in Proc. IEEE International Conference on Computer Vision (ICCV), 2017.